

Guía de migración de SwagDoc: Swagger 2.0 a OpenAPI 3.2.1

De `svSwagger2` a `svOpenApi3`: qué cambió en la especificación, qué cambiar en su código Delphi, qué archivos desplegar y cómo validar el resultado.

Biblioteca SwagDoc - versión que introduce `TSwagDoc.SpecVersion` (septiembre de 2026). Solo RTL de Delphi, utilizable en aplicaciones VCL, FMX, de consola y de servidor.

Contenido

1. Descripción general
2. El modelo de versiones
3. Qué cambió en la especificación
4. Compatibilidad y cambios incompatibles
5. Migración paso a paso
6. Migración del código
7. Qué no recibe un documento Swagger 2.0
8. Despliegue
9. Validación y pruebas
10. Limitaciones conocidas
11. Checklist de migración
12. Referencias

1. Descripción general

SwagDoc ahora escribe el mismo modelo de objetos como un documento Swagger 2.0 (`swagger.json`) o como un documento OpenAPI 3 (`openapi.json`). La familia se selecciona con la propiedad `SpecVersion` de `TSwagDoc`.

Puntos clave antes de comenzar:

- `SpecVersion` tiene como valor predeterminado `svSwagger2`. Una aplicación que no toca la nueva propiedad sigue produciendo el mismo documento Swagger 2.0 que producía antes.
- Una sola línea cambia la salida a OpenAPI 3.2.1: `vSwagDoc.SpecVersion := svOpenApi3;`.
- Las clases y propiedades de Swagger 2.0 siguen funcionando cuando el documento se genera como OpenAPI 3. Host, base path, schemes, consumes, produces y los parámetros body y formData se traducen automáticamente.
- Los objetos de OpenAPI 3 (servidores, cuerpos de solicitud, tipos de medio, links, callbacks, webhooks, requisitos de seguridad con ámbitos (scopes), extensiones y todos los componentes reutilizables) están disponibles en el modelo de objetos. Se traducen u omiten cuando se genera un documento Swagger 2.0.
- `LoadFromFile` lee documentos Swagger 2.0 y OpenAPI 3.0, 3.1 y 3.2, por lo que un `swagger.json` existente se puede convertir.
- Los documentos OpenAPI 3.2 requieren Swagger UI 5. La carpeta `Deploy` ahora tiene una carpeta por familia.

Respuestas rápidas

Pregunta	Respuesta
¿Tengo que cambiar mi código después de actualizar la biblioteca?	No, a menos que su código tenga alguno de los casos a nivel de código fuente de la sección 4.3 (enumeraciones ampliadas, definiciones de seguridad personalizadas o una lista explícita de units en un package).
¿Cambia mi salida Swagger 2.0?	Los documentos construidos en código mantienen la misma salida. Los documentos cargados desde un archivo <code>swagger.json</code> ahora se escriben con más información (sección 4.2).
¿Cuál es la migración mínima?	Asigne <code>SpecVersion := svOpenApi3</code> y publique <code>openapi.json</code> con los archivos de Swagger UI 5 de <code>Deploy\OpenApi3</code> .
¿Puedo publicar ambas versiones?	Sí. Genere el documento dos veces a partir del mismo modelo, cambiando <code>SpecVersion</code> (sección 6.15).
¿Qué versión de OpenAPI se escribe?	La versión 3.x más reciente compatible con SwagDoc, actualmente 3.2.1. <code>SwaggerVersion</code> devuelve el valor exacto.

Estrategias de migración

1. **Solo el cambio.** Mantenga el modelo Swagger 2.0 y asigne `SpecVersion := svOpenApi3`. SwagDoc lo traduce todo. Se recomienda como primer paso.
2. **Período de transición.** Publique `swagger.json` y `openapi.json` en paralelo mientras los consumidores de la API migran al nuevo documento.
3. **Adopción completa.** Reemplace las construcciones de Swagger 2.0 por los objetos de OpenAPI 3 (servidores, cuerpos de solicitud, contenido por tipo de medio, requisitos de seguridad con ámbitos) y comience a usar los objetos que solo existen en OpenAPI 3.

2. El modelo de versiones

SpecVersion	Campo raíz del documento	Nombre de archivo predeterminado	Dialecto de esquema
svSwagger2 (predeterminado)	"swagger": "2.0"	swagger.json	Objeto de esquema de Swagger 2.0 (subconjunto de JSON Schema draft 4)
svOpenApi3	"openapi": "3.2.1"	openapi.json	JSON Schema 2020-12

`SpecVersion` designa una **familia** de la especificación, no una versión individual:

- Cuando se admite una versión 3.x más reciente, `svOpenApi3` comienza a escribirla y su código no cambia.
- Solo se agrega un nuevo valor para una nueva familia (OpenAPI 4) o para una versión incompatible con los documentos ya producidos por su familia.
- La versión exacta escrita en el documento la devuelve `SwaggerVersion` ("2.0" o "3.2.1").

uses

```
Swagger.Common.Types, Swagger.Doc;  
  
vSwaggerDoc.SpecVersion := svOpenApi3;  
vSwaggerDoc.GenerateSwaggerJson;           // vSwaggerDoc.SwaggerVersion = '3.2.1'  
vSwaggerDoc.SwaggerFilesFolder := 'C:\MyApi\Help';  
vSwaggerDoc.SaveSwaggerJsonToFile;         // escribe C:\MyApi\Help\openapi.json
```

El nombre del archivo sigue a la familia, a menos que se asigne `SwaggerFileName`. Asigne `SwaggerFileName := 'swagger.json'` cuando la URL publicada no deba cambiar.

3. Qué cambió en la especificación

3.1 Estructura del documento: Swagger 2.0 comparado con OpenAPI 3.2.1

Área	Swagger 2.0	OpenAPI 3.2.1	Modelo de objetos de SwagDoc
Campo de versión	swagger: "2.0"	openapi: "3.2.1"	SpecVersion
Identidad del documento	ninguna	\$self, jsonSchemaDialect	SelfUri, JsonSchemaDialect
Servidor de destino	host, basePath, schemes	servers con plantillas de URL, variables y nombre	Servers, AddServer, TSwagServer.Name
Tipos de medio	consumes, produces	mapa content en cada cuerpo de la solicitud, respuesta, parámetro y encabezado	AddMediaType de RequestBody, TSwagResponse, TSwagRequestParameter, TSwagHeaders
Carga útil de la solicitud	parámetros body y formData	requestBody	TSwagPathOperation.RequestBody
Objetos reutilizables	definitions, parameters, responses, securityDefinitions	components con schemas, responses, parameters, examples, requestBodies, headers, securitySchemes, links, callbacks, pathItems, mediaTypes	Definitions, Responses, Parameters, Examples, RequestBodies, Headers, SecurityDefinitions, Links, Callbacks, PathItems, MediaTypes
Esquemas	Objeto de esquema, extensión x-nullable	JSON Schema 2020-12 (arreglos type, límites exclusivos numéricos, arreglo examples)	Convertidos automáticamente
Ubicaciones de parámetros	query, header, path, formData, body	query, header, path, cookie, querystring	rpiCookie, rpiQueryString
Serialización de parámetros	collectionFormat	style, explode, allowReserved, content	Style, Explode, AllowReserved, Content
Métodos HTTP	get, put, post, delete, options, head, patch	agrega trace, query y additionalOperations	ohvTrace, ohvQuery, AddAdditionalOperation
Ejemplos	examples de una respuesta, por tipo MIME	Example Object con dataValue, serializedValue, externalValue	TSwagExample
Encabezados	tipo y formato	Header Object con schema, content, examples, required, deprecated	TSwagHeaders

Área	Swagger 2.0	OpenAPI 3.2.1	Modelo de objetos de SwagDoc
Relaciones entre operaciones	ninguna	links y callbacks	<code>TSwagLink</code> , <code>TSwagCallback</code>
Eventos enviados por la API	ninguno	webhooks	<code>Webhooks</code>
Esquemas de seguridad	basic, apiKey, oauth2 (un flujo)	http, apiKey (también cookie), oauth2 (varios flujos), openIdConnect, mutualTLS	<code>TSwagSecurityDefinitionHttp</code> , <code>...ApiKey</code> , <code>...OAuth2</code> , <code>...OpenIdConnect</code> , <code>...MutualTls</code>
Requisitos de seguridad	ámbitos solo para OAuth2	ámbitos para todos los esquemas, combinaciones AND y OR	<code>TSwagSecurityRequirement</code>
Etiquetas	name, description, externalDocs	agrega summary, parent (etiquetas anidadas) y kind	<code>Summary</code> , <code>Parent</code> , <code>Kind</code>
Extensiones	campos x-	campos x- en todos los objetos	Propiedad <code>Extensions</code>

3.2 Novedades por versión

OpenAPI 3.0

- Servidores, componentes, cuerpos de solicitud, contenido y tipos de medio, codificación de contenidos multipart y de formulario.
- Callbacks, links, parámetros de cookie, `style` y `explode`.
- Esquemas de seguridad HTTP y OpenID Connect, el Example Object y el Header Object con esquema.

OpenAPI 3.1

- Alineación completa con JSON Schema 2020-12: los valores nulos se escriben como arreglos `type`, `exclusiveMinimum` y `exclusiveMaximum` son números, y los esquemas aceptan un arreglo `examples`.
- Webhooks, `info.summary`, `license.identifier` (SPDX), `components.pathItems`, el esquema de seguridad mutual TLS y `jsonSchemaDialect`.
- `summary` y `description` junto a un `$ref` sobrescriben los valores referenciados, y los requisitos de seguridad aceptan roles para todos los esquemas.

OpenAPI 3.2

- `$self`, el método QUERY, `additionalOperations` y la ubicación de parámetro `queryString`.
- `summary`, `parent` y `kind` de las etiquetas, `name` del servidor y `summary` de la respuesta.
- `itemSchema` del tipo de medio para tipos de medio secuenciales (JSON Lines, server-sent events), `prefixEncoding`, `itemEncoding` y `components.mediaTypes`.
- `dataValue` y `serializedValue` del ejemplo.
- Flujo OAuth2 `deviceAuthorization`, `oauth2MetadataUrl` y el campo `deprecated` de los esquemas de seguridad.

OpenAPI 3.2.1 es una versión de corrección de 3.2 con aclaraciones y correcciones. No agrega objetos nuevos.

4. Compatibilidad y cambios incompatibles

4.1 Qué se mantiene igual

- `TSwagDoc` se inicia con `SpecVersion = svSwagger2`, por lo que `GenerateSwaggerJson` escribe Swagger 2.0 como antes.
- Los documentos contruidos en código por las demos del repositorio (`SampleApi`, `GenerateSwaggerJsonFromCode`) son idénticos byte a byte a la salida de la versión anterior. El generador de Swagger 2.0 solo incorporó rutas de código para propiedades que antes no existían.
- Las units utilizadas por las aplicaciones existentes siguen disponibles. `Swag.Doc.Path` y `Swag.Doc.Path.Operation.ResponseHeaders` siguen declarando `TSwagPath` y `TSwagHeaders` como alias de las clases que se movieron (sección 4.3).
- `TJsonSchema` y sus campos producen los mismos esquemas. La propiedad `Nullable` y el `Format` de los campos de tipo string son nuevos y opcionales.

4.2 Cambios en la salida para documentos cargados desde un archivo swagger.json

Cuando se carga un archivo Swagger 2.0 con `LoadFromFile` y se genera nuevamente como Swagger 2.0, la versión anterior perdía parte del documento. Son correcciones, pero el archivo generado es diferente:

Contenido del archivo cargado	Versión anterior	Versión actual
<code>securityDefinitions</code>	No se escribía	Se escribe
<code>security</code> del documento y de las operaciones	No se escribía, o se escribía sin ámbitos	Se escribe con los ámbitos (por ejemplo <code>["write:pets", "read:pets"]</code>)
<code>externalDocs</code> del documento	No se escribía	Se escribe
<code>items</code> y <code>enum</code> de parámetros de tipo arreglo	Se perdían	Se escriben
<code>allowEmptyValue</code>	Se escribía como <code>true</code> para parámetros query que no lo declaraban	Se escribe solo cuando está declarado
<code>responses</code> del documento y extensiones <code>x-</code>	Se perdían	Se escriben

Si un proceso posterior compara el archivo generado con una copia almacenada, actualice la copia almacenada después de la actualización.

4.3 Cambios a nivel de código fuente que pueden requerir actualizar el código

Enumeraciones ampliadas

Enumeración	Valores agregados
<code>TSwagSecurityDefinitionType</code>	<code>ssdHttp</code> , <code>ssdOpenIdConnect</code> , <code>ssdMutualTls</code>
<code>TSwagRequestParameterInLocation</code>	<code>rpiCookie</code> , <code>rpiQueryString</code>
<code>TSwagPathTypeOperation</code>	<code>ohvQuery</code>

Enumeración	Valores agregados
TSwagSecurityDefinitionApiKeyInLocation	Cookie
Tipos nuevos	TSwagVersion, TSwagRequestParameterStyle, TSwagOAuth2FlowType

Revise en su código:

- Arreglos constantes indexados por alguna de estas enumeraciones, como `array[TSwagRequestParameterInLocation] of string`. Ya no compilan (E2072, el número de elementos difiere) y necesitan los nuevos valores.
- Bucles de `Low` a `High` de estas enumeraciones. Ahora recorren los nuevos valores; por ejemplo, un generador de código cliente recibiría el método `query`.
- Sentencias `case` sin rama `else` que deban manejar todos los valores.

Definiciones de seguridad personalizadas

`TSwagSecurityDefinition` incorporó sobrecargas y nuevos miembros virtuales. Una clase que hereda de ella sigue compilando, pero agregue la directiva `overload` para que las nuevas sobrecargas sigan visibles, y llame a `inherited` en su destructor, porque la clase base ahora es propietaria del objeto `Extensions`.

```
// Versión anterior
TMySecurityDefinition = class(TSwagSecurityDefinition)
protected
    function GetTypeSecurity: TSwagSecurityDefinitionType; override;
public
    function GenerateJsonObject: TJSONObject; override;
    procedure Load(pJson: TJSONObject); override;
end;

// Versión actual
TMySecurityDefinition = class(TSwagSecurityDefinition)
protected
    function GetTypeSecurity: TSwagSecurityDefinitionType; override;
public
    function GenerateJsonObject: TJSONObject; overload; override;
    function GenerateJsonObject(const pVersion: TSwagVersion): TJSONObject; overload; override;
    procedure Load(pJson: TJSONObject); overload; override;
    procedure Load(pJson: TJSONObject; const pVersion: TSwagVersion); overload; override;
    function SupportsVersion(const pVersion: TSwagVersion): Boolean; override;
end;
```

- `GenerateJsonObject(pVersion)` y `Load(pJson, pVersion)` llaman de forma predeterminada a los métodos sin versión, así que sobrescríbalos solo cuando la representación en OpenAPI 3 sea diferente.
- Devuelva `False` desde `SupportsVersion` para una familia que no pueda representar el esquema. El esquema y los requisitos que lo usan se eliminan entonces de esa familia.
- `Description`, `Deprecated` y `Extensions` están disponibles para todos los esquemas.

Clases que se movieron a otras units

Clase	Unit anterior	Unit actual	¿La unit anterior sigue funcionando?
TSwagPath	Swag.Doc.Path	Swag.Doc.Path.Operation	Sí, Swag.Doc.Path declara un alias
TSwagHeaders	Swag.Doc.Path.Operation.ResponseHeaders	Swag.Doc.Path.Operation.Content	Sí, ResponseHeaders declara un alias

Los alias son el mismo tipo, por lo que `is`, `as` y las variables existentes siguen funcionando.

Units nuevas

Las aplicaciones que agregan la carpeta `Source` al library path no necesitan nada más. Los packages o proyectos que enumeran explícitamente las units de SwaggerDoc deben agregar:

Unit	Contenido
Swag.Common.Json	Funciones auxiliares de lectura de JSON
Swag.Doc.Extensions	Extensiones de la especificación (<code>TSwagExtensions</code>)
Swag.Doc.Example	Example Object (<code>TSwagExample</code>)
Swag.Doc.Link	Link Object (<code>TSwagLink</code>)
Swag.Doc.SecurityRequirement	Requisitos de seguridad con ámbitos
Swag.Doc.Server	Server Object y variables de servidor
Swag.Doc.Path.Operation.Content	Tipos de medio, codificaciones y encabezados
Swag.Doc.Path.Operation.RequestBody	Request Body Object
Swag.Doc.SecurityDefinitionHttp	Esquema de seguridad HTTP (basic, bearer y otros)
Swag.Doc.SecurityDefinitionOpenIdConnect	Esquema de seguridad OpenID Connect
Swag.Doc.SecurityDefinitionMutualTls	Esquema de seguridad mutua TLS
Swag.Doc.JsonConverter	Conversión de referencias y palabras clave de esquema entre las familias
Swag.Doc.OpenApi.Generator	Escritor de OpenAPI 3
Swag.Doc.OpenApi.Loader	Lector de OpenAPI 3

Otros comportamientos a tener en cuenta

- `LoadFromFile` asigna `SpecVersion` según el archivo cargado. Después de cargar un archivo OpenAPI 3, asigne `SpecVersion := svSwagger2` si se espera un documento Swagger 2.0.
- `Extensions.Add` lanza `ESwagErrorExtensionName` cuando el nombre no comienza con `x-`.
- Las propiedades que reciben valores JSON (`Example`, `DataValue`, `Value`, `Items`, `RequestBody` de un link, `ItemEncoding`) toman posesión del objeto asignado. No lo libere.
- `Deploy\index.html`, `Deploy\swagger.json` y los archivos de Swagger UI de la raíz de `Deploy` se movieron a `Deploy\Swagger2`. Actualice los scripts que los copian.

5. Migración paso a paso

1. Actualice el código fuente de SwagDoc y recompile. Si instala `Source\SwagDoc.dpk`, recompile el package y elimine los archivos `.dcu` y `.bp1` antiguos de sus carpetas de salida.
2. Corrija los casos a nivel de código fuente de la sección 4.3, si los hay.
3. Compile la aplicación sin cambiar `SpecVersion` y compare el `swagger.json` generado con el anterior (sección 9.1).
4. Asigne `SpecVersion := svOpenApi3`, genere `openapi.json` y válidelo (sección 9.2).
5. Despliegue los archivos de Swagger UI 5 de `Deploy\OpenApi3` y configure la URL del documento (sección 8).
6. Pruebe la página en el navegador: operaciones, esquemas, el diálogo Authorize y las solicitudes de Try it out.
7. Opcionalmente, publique ambos documentos durante un período de transición (sección 6.15).
8. Migre el modelo a los objetos de OpenAPI 3 donde mejoren la documentación (sección 6).
9. Actualice los consumidores de la API: generadores de clientes, gateways y pruebas de contrato que leen el documento.

6. Migración del código

Cada tema muestra la construcción de Swagger 2.0, que sigue siendo válida, y la construcción de OpenAPI 3. Las cláusulas `uses` enumeran solo las units que necesita el ejemplo.

6.1 Cambiar la versión y mantener la URL

```
vSwaggerDoc.SpecVersion := svOpenApi3;
vSwaggerDoc.GenerateSwaggerJson;
vSwaggerDoc.SwaggerFilesFolder := 'C:\MyApi\Help';
vSwaggerDoc.SwaggerFileName := 'swagger.json'; // opcional: mantener el nombre de archivo anterior
vSwaggerDoc.SaveSwaggerJsonToFile;
```

6.2 Host, base path y schemes se convierten en servidores

Construcción de Swagger 2.0:

```
vSwaggerDoc.Host := 'api.example.com';
vSwaggerDoc.BasePath := '/v1';
vSwaggerDoc.Schemes := [tpsHttps];
```

Cuando la lista `Servers` está vacía, OpenAPI 3 recibe un servidor por cada scheme:

`https://api.example.com/v1`. Con servidores:

```
uses
    Swag.Doc.Server;

var
    vServer: TSwaggerServer;
    vVariable: TSwaggerServerVariable;

begin
    vServer := vSwaggerDoc.AddServer('https://{environment}.example.com/v1', 'Cloud server');
    vServer.Name := 'cloud';
    vVariable := vServer.AddVariable('environment', 'api', 'The environment of the API. ');
    vVariable.Enum.Add('api');
    vVariable.Enum.Add('sandbox');

    vSwaggerDoc.AddServer('http://localhost:8080/v1', 'Local development server');
end;
```

Los servidores no se escriben en Swagger 2.0. Si también publica `swagger.json`, mantenga `Host`, `BasePath` y `Schemes`. Cuando `Servers` tiene elementos, OpenAPI 3 ignora esas tres propiedades. `TSwagPath` y `TSwagPathOperation` también tienen una lista `Servers`.

6.3 Los parámetros body se convierten en un cuerpo de la solicitud

Construcción de Swagger 2.0:

```

vParameter := TSwagRequestParameter.Create;
vParameter.Name := 'employee';
vParameter.InLocation := rpiBody;
vParameter.Required := True;
vParameter.Schema.Name := 'Employee';
vOperation.Parameters.Add(vParameter);
vOperation.Consumes.Add('application/json');

```

OpenAPI 3 recibe un `requestBody` con un tipo de medio por cada tipo MIME consumido. Con el objeto de cuerpo de la solicitud, cada tipo de medio puede tener su propio esquema:

```

vOperation.RequestBody.Description := 'The employee data.';
vOperation.RequestBody.Required := True;
vOperation.RequestBody.AddMediaType('application/json').Schema.Name := 'Employee';
vOperation.RequestBody.AddMediaType('application/xml').Schema.Name := 'Employee';

```

En Swagger 2.0, el cuerpo de la solicitud se escribe como un parámetro body con el esquema del primer tipo de medio, y sus tipos de medio van a `consumes`. Esto ocurre solo cuando la operación no tiene ningún parámetro body ni formData.

6.4 Los parámetros formData se convierten en un cuerpo de la solicitud de formulario o multipart

Construcción de Swagger 2.0:

```

vParameter := TSwagRequestParameter.Create;
vParameter.Name := 'file';
vParameter.InLocation := rpiFormData;
vParameter.TypeParameter := stpFile;
vParameter.Required := True;
vOperation.Parameters.Add(vParameter);

```

OpenAPI 3 recibe un cuerpo de la solicitud `multipart/form-data` con un esquema de objeto, y el tipo `file` se convierte en `type: string, format: binary`. Con el objeto de cuerpo de la solicitud:

uses

```
Swag.Doc.Path.Operation.Content;
```

var

```
vMediaType: TSwagMediaType;
```

begin

```
vMediaType := vOperation.RequestBody.AddMediaType('multipart/form-data');
```

```
vMediaType.Schema.JsonSchema := TJSONObject.ParseJSONValue(
```

```
'{"type":"object","properties":{"title":{"type":"string"},"'
```

```
'"file":{"type":"string","contentMediaType":"application/pdf"},"required":["file"]}') as
```

```
TJSONObject;
```

```
vMediaType.AddEncoding('file').ContentType := 'application/pdf';
```

end;

En Swagger 2.0, un cuerpo de la solicitud de formulario o multipart se escribe como parámetros formData. Una propiedad con `format: binary` o `contentMediaType` se convierte en un parámetro `file` y una propiedad `object` se convierte en un parámetro `string`.

6.5 Produce y los esquemas de respuesta se convierten en content

Construcción de Swagger 2.0:

```
vOperation.Produces.Add('application/json');
```

```
vResponse := TSwagResponse.Create;
```

```
vResponse.StatusCode := '200';
```

```
vResponse.Description := 'The employee.';
```

```
vResponse.Schema.Name := 'Employee';
```

```
vOperation.Responses.Add(vResponse.StatusCode, vResponse);
```

OpenAPI 3 recibe el esquema en el `content` de la respuesta para cada tipo de medio producido (`application/json` cuando no se produce ninguno). Con contenido por tipo de medio:

```
vResponse.AddMediaType('application/json').Schema.Name := 'Employee';
```

```
vResponse.AddMediaType('application/jsonl').ItemSchema.Name := 'Employee'; // un empleado por línea
```

- Cuando `Content` tiene elementos, `Produces` y `Schema` no se usan para esa respuesta en OpenAPI 3.
- En Swagger 2.0 se escribe el esquema del primer tipo de medio cuando `Schema` está vacío.
- `Summary` y `Links` de una respuesta se escriben solo en OpenAPI 3.

6.6 Encabezados de respuesta

La construcción de Swagger 2.0 funciona en ambas familias:

```
vHeader := vResponse.AddHeader('X-Rate-Limit-Remaining');
```

```
vHeader.Description := 'The number of requests left in the current period.';
```

```
vHeader.ValueType := 'integer';
```

El Header Object de OpenAPI 3 acepta un esquema, ejemplos, `Required`, `Deprecated`, `Style`, `Explode` y `Content`:

```
vHeader := vResponse.AddHeader('X-Rate-Limit-Remaining');
vHeader.Description := 'The number of requests left in the current period.';
vHeader.Schema.JsonSchema := TJSONObject.Create.AddPair('type', 'integer');
vHeader.Example := TJSONNumber.Create(99);
```

En Swagger 2.0, `ValueType` y `Format` se toman del esquema cuando están vacíos. Los encabezados que son referencias no se escriben en Swagger 2.0.

6.7 Ejemplos

Construcción de Swagger 2.0:

```
vResponse.Examples.Add('application/json', TJSONObject.ParseJSONValue('{ "id":42,"name":"John Smith"}')
as TJSONObject);
```

OpenAPI 3 recibe un Example Object por cada entrada, dentro del contenido de cada tipo de medio producido. Con el Example Object:

```
uses
    Swag.Doc.Example, Swag.Doc.Path.Operation.Content;

var
    vMediaType: TSwagMediaType;
    vExample: TSwagExample;

begin
    vMediaType := vResponse.AddMediaType('application/json');
    vMediaType.Schema.Name := 'Employee';
    vExample := vMediaType.AddExample('johnSmith');
    vExample.Summary := 'An employee';
    vExample.DataValue := TJSONObject.ParseJSONValue('{ "id":42,"name":"John Smith"}');

    vMediaType := vResponse.AddMediaType('text/csv');
    vMediaType.AddExample('csv').SerializedValue := 'id,name'#10'42,John Smith';

end;
```

- Use solo uno de `DataValue`, `SerializedValue` y `ExternalValue`. `Value` todavía se acepta, pero no debe combinarse con `DataValue` ni con `SerializedValue`.
- Los ejemplos reutilizables van a `vSwagDoc.Examples` y se referencian con `Ref := '#/components/examples/Name'`.
- En Swagger 2.0, el primer ejemplo de cada tipo de medio se escribe en los `examples` de la respuesta.

6.8 Componentes reutilizables y referencias

Las referencias escritas para Swagger 2.0 se reescriben en ambas direcciones:

Referencia en Swagger 2.0	Referencia en OpenAPI 3
<code>#/definitions/Name</code>	<code>#/components/schemas/Name</code>
<code>#/parameters/Name</code>	<code>#/components/parameters/Name</code> , o <code>#/components/requestBodies/Name</code> para parámetros body y formData
<code>#/responses/Name</code>	<code>#/components/responses/Name</code>

Prefiera `Schema.Name` a las referencias escritas a mano: SwagDoc escribe la ruta correcta para cada familia. Los componentes que no existen en Swagger 2.0 (ejemplos, encabezados, links, callbacks, path items y tipos de medio) deben referenciarse con la ruta de OpenAPI 3.

```

vResponse := TSwagResponse.Create;
vResponse.Name := 'notFound'; // clave dentro de components/responses
vResponse.Description := 'The employee was not found.';
vResponse.AddMediaType('application/problem+json').Schema.Name := 'Problem';
vSwagDoc.Responses.Add(vResponse);

vResponse := TSwagResponse.Create;
vResponse.StatusCode := '404';
vResponse.Ref := '#/components/responses/notFound';
vResponse.Summary := 'Unknown employee'; // sobrescribe el summary referenciado
vOperation.Responses.Add(vResponse.StatusCode, vResponse);

```

Propiedad de TSwagDoc	Campo de <code>components</code>	Clave de cada elemento
Definitions	schemas	Name
Responses	responses	Name
Parameters	parameters	Name
Examples	examples	Name
RequestBodies	requestBodies	Name
Headers	headers	Name
SecurityDefinitions	securitySchemes	SchemeName
Links	links	Name
Callbacks	callbacks	Name
PathItems	pathItems	Uri
MediaTypes	mediaTypes	Name

La `Description` escrita junto con la `Ref` de un parámetro, cuerpo de la solicitud, respuesta o encabezado sobrescribe la referenciada en OpenAPI 3, y lo mismo ocurre con el `Summary` de una respuesta.

6.9 Esquemas y requisitos de seguridad

Autenticación basic y bearer

```
// Construcción de Swagger 2.0, todavía válida: se escribe como http/basic en OpenAPI 3
vBasic := TSwagSecurityDefinitionBasic.Create;
vBasic.SchemeName := 'basicAuth';
vSwagDoc.SecurityDefinitions.Add(vBasic);

// Alternativa de Swagger 2.0 para tokens bearer
vApiKey := TSwagSecurityDefinitionApiKey.Create;
vApiKey.SchemeName := 'bearerAuth';
vApiKey.InLocation := kilHeader;
vApiKey.Name := 'Authorization';
vSwagDoc.SecurityDefinitions.Add(vApiKey);

// OpenAPI 3: esquema HTTP bearer (se escribe como La API key anterior en Swagger 2.0)
vBearer := TSwagSecurityDefinitionHttp.Create;
vBearer.SchemeName := 'bearerAuth';
vBearer.Scheme := 'bearer';
vBearer.BearerFormat := 'JWT';
vSwagDoc.SecurityDefinitions.Add(vBearer);
```

Flujos OAuth2

Construcción de Swagger 2.0, con un flujo:

```
vOAuth2 := TSwagSecurityDefinitionOAuth2.Create;
vOAuth2.SchemeName := 'petstore_auth';
vOAuth2.Flow := 'accessCode';
vOAuth2.AuthorizationUrl := 'https://auth.example.com/authorize';
vOAuth2.TokenUrl := 'https://auth.example.com/token';
vOAuth2.AddScope('read:pets', 'Read your pets');
vSwagDoc.SecurityDefinitions.Add(vOAuth2);
```

Los nombres de los flujos se traducen entre las familias:

Swagger 2.0	OpenAPI 3	TSwagOAuth2FlowType
implicit	implicit	oftImplicit
password	password	oftPassword
application	clientCredentials	oftClientCredentials
accessCode	authorizationCode	oftAuthorizationCode
ninguno	deviceAuthorization	oftDeviceAuthorization

Construcción de OpenAPI 3, con varios flujos:

uses

```
Swag.Doc.SecurityDefinitionOAuth2;
```

var

```
vOAuth2: TSwagSecurityDefinitionOAuth2;  
vFlow: TSwagSecurityDefinitionOAuth2Flow;
```

begin

```
vOAuth2 := TSwagSecurityDefinitionOAuth2.Create;  
vOAuth2.SchemeName := 'oauth2Auth';  
vOAuth2.OAuth2MetadataUrl := 'https://auth.example.com/.well-known/oauth-authorization-server';  
  
vFlow := vOAuth2.AddFlow(oftAuthorizationCode);  
vFlow.AuthorizationUrl := 'https://auth.example.com/authorize';  
vFlow.TokenUrl := 'https://auth.example.com/token';  
vFlow.AddScope('employees:write', 'Creates and updates the employees');  
  
vFlow := vOAuth2.AddFlow(oftClientCredentials);  
vFlow.TokenUrl := 'https://auth.example.com/token';  
vFlow.AddScope('employees:read', 'Reads the employees');  
  
vSwagDoc.SecurityDefinitions.Add(vOAuth2);
```

end;

En Swagger 2.0 se escribe el primer flujo que existe en Swagger 2.0. Cuando `Flows` está vacío, se usan las propiedades de flujo único en ambas familias.

Requisitos de seguridad

Construcción de Swagger 2.0, sin ámbitos:

```
vOperation.Security.Add('petstore_auth');
```

Construcción de OpenAPI 3, con ámbitos y combinaciones:

uses

```
Swag.Doc.SecurityRequirement;
```

var

```
vRequirement: TSwagSecurityRequirement;
```

begin

```
// documento: token bearer O el ámbito de Lectura de OAuth2
```

```
vSwagDoc.AddSecurityRequirement.AddScheme('bearerAuth', []);
```

```
vSwagDoc.AddSecurityRequirement.AddScheme('oauth2Auth', ['employees:read']);
```

```
// operación: el ámbito de escritura Y un certificado de cliente
```

```
vRequirement := vOperation.AddSecurityRequirement;
```

```
vRequirement.AddScheme('oauth2Auth', ['employees:write']);
```

```
vRequirement.AddScheme('mutualTlsAuth', []);
```

```
// operación sin autenticación: security: []
```

```
vHealthOperation.DisableSecurity := True;
```

end;

Cada `AddSecurityRequirement` es una alternativa (OR) y los esquemas de un requisito son todos obligatorios (AND). Los requisitos se escriben en ambas familias. El orden de precedencia es:

Nivel	Primera opción	Segunda opción	Tercera opción
Documento	<code>SecurityRequirements</code>	<code>DisableSecurity</code> escribe []	<code>GlobalSecurityFromDefinitions</code> escribe cada definición de seguridad como una alternativa sin ámbitos
Operación	<code>SecurityRequirements</code>	Lista <code>Security</code> , sin ámbitos	<code>DisableSecurity</code> escribe []

`GlobalSecurityFromDefinitions` es `False` de forma predeterminada, por lo que un documento sin requisitos no escribe un `security` propio y cada operación conserva la seguridad que declara. Defínalo como `True` para que cada esquema sea una forma alternativa de llamar a toda la API.

Esquemas en un documento Swagger 2.0

Esquema en el modelo	Salida en Swagger 2.0
HTTP basic	<code>type: basic</code>
HTTP bearer u otro esquema HTTP	<code>type: apiKey, in: header, name: Authorization</code>
OpenID Connect	<code>type: apiKey</code> en el encabezado <code>Authorization</code> con la extensión <code>x-openIdConnectUrl</code>
API key en una cookie	Se elimina, junto con los requisitos que la usan
Mutual TLS	Se elimina, junto con los requisitos que lo usan
OAuth2 con solo el flujo device authorization	Se elimina, junto con los requisitos que lo usan
<code>Deprecated</code> de cualquier esquema	No se escribe

6.10 Esquemas y valores nulos

```
vPhone := vSchema.AddField<string>('phone', 'The employee phone number.');
```

```
vPhone.Nullable := True;
```

Las palabras clave de esquema se convierten para la familia que se está generando, incluidos los esquemas JSON asignados a mano a `JsonSchema`:

Construcción	Salida en Swagger 2.0	Salida en OpenAPI 3.2.1
Campo <code>Nullable</code> o <code>nullable: true</code>	No se escribe, o <code>x-nullable: true</code> cuando <code>WriteNullableExtension</code> es <code>True</code>	Arreglo <code>type</code> que incluye <code>"null"</code>
Referencia que admite nulos	Solo la referencia, o <code>allOf</code> con <code>x-nullable</code> cuando <code>WriteNullableExtension</code> es <code>True</code>	<code>anyOf</code> con la referencia y <code>type: "null"</code>
<code>x-nullable: true</code> leído de un documento Swagger 2.0	Se mantiene	Arreglo <code>type</code> que incluye <code>"null"</code>
<code>exclusiveMinimum</code> y <code>exclusiveMaximum</code>	Booleanos, junto con <code>minimum</code> y <code>maximum</code>	Límites numéricos
<code>type: file</code>	Se mantiene	<code>type: string</code> , <code>format: binary</code>
Arreglo <code>examples</code> del esquema	Primer elemento como <code>example</code>	Se mantiene

Un documento OpenAPI 3.0 cargado con `LoadFromFile` se actualiza cuando se genera nuevamente.

6.11 Parámetros

Propiedad	Uso	Salida en Swagger 2.0
<code>InLocation := rpiCookie</code>	Parámetro enviado en una cookie	No se escribe
<code>InLocation := rpiQueryString</code>	Toda la query string como un único parámetro, con <code>Content</code> (predeterminado <code>application/x-www-form-urlencoded</code>)	No se escribe
<code>Style</code> , <code>Explode</code> , <code>AllowReserved</code>	Serialización de arreglos y objetos	No se escribe
<code>Deprecated</code>	Parámetro en proceso de retiro	No se escribe
<code>Example</code> , <code>Examples</code>	Ejemplos del valor	No se escribe
<code>Content</code>	Valores complejos, por ejemplo JSON en un parámetro query	No se escribe
<code>Schema</code>	Cualquier esquema, en cualquier ubicación	Se escribe solo para parámetros body
<code>Description</code> con <code>Ref</code>	Sobrescribe la descripción referenciada	No se escribe

```

vParameter := TSwaggerRequestParameter.Create;
vParameter.Name := 'department';
vParameter.InLocation := rpiQuery;
vParameter.TypeParameter := stpString;
vParameter.Style := rpsForm;
vParameter.Explode := True;
vExample := vParameter.AddExample('sales');
vExample.Summary := 'Sales department';
vExample.DataValue := TJSONString.Create('sales');
vOperation.Parameters.Add(vParameter);

```

6.12 Objetos que solo existen en OpenAPI 3

Funcionalidad	Código	Salida en Swagger 2.0
URI del documento	<code>vSwagDoc.SelfUri := 'https://api.example.com/v1/openapi.json'</code>	No se escribe
Dialecto de esquema	<code>vSwagDoc.JsonSchemaDialect := '...'</code>	No se escribe
Resumen de info, licencia SPDX	<code>Info.Summary, Info.License.Identifier</code>	El resumen no se escribe; el identificador se convierte en la URL de la licencia cuando <code>Url</code> está vacío
Etiquetas anidadas	<code>Tag.Summary, Tag.Parent, Tag.Kind</code>	No se escribe
Método QUERY	<code>vPath.AddOperation(ohvQuery)</code>	La operación no se escribe
Otros métodos	<code>vPath.AddAdditionalOperation('LINK')</code>	No se escribe
Path item reutilizable	<code>vSwagDoc.PathItems</code> y <code>vPath.Ref := '#/components/pathItems/Name'</code>	El path item referenciado se escribe en línea
Webhooks	<code>vSwagDoc.Webhooks.Add(vWebhook)</code>	No se escribe
Callbacks	<code>vOperation.AddCallback('Name').AddPathItem('{ \$request.body#/callbackUrl}')</code>	No se escribe
Links	<code>vResponse.AddLink('Name')</code>	No se escribe
Tipos de medio secuenciales	<code>vMediaType.ItemSchema</code>	No se escribe
Codificaciones	<code>AddEncoding, AddPrefixEncoding, ItemEncoding</code>	No se escribe
Extensiones de la especificación	<code>Extensions.Add('x-name', vValue)</code>	Se escribe para los objetos que existen en Swagger 2.0

```

vLink := vResponse.AddLink('GetEmployeeById');
vLink.OperationId := 'getEmployee';
vLink.AddParameter('id', '$response.body#/id');

vCallback := vOperation.AddCallback('exportCompleted');
vCallbackOperation := vCallback.AddPathItem({'$request.body#/callbackUrl'}).AddOperation(ohvPost);
vCallbackOperation.RequestBody.AddMediaType('application/json').Schema.Name := 'ExportResult';

vWebhook := TSwagPath.Create;
vWebhook.Uri := 'employeeHired';
vWebhook.AddOperation(ohvPost).RequestBody.AddMediaType('application/json').Schema.Name := 'Employee';
vSwagDoc.Webhooks.Add(vWebhook);

```

6.13 Convertir un archivo swagger.json existente

```

procedure ConvertToOpenApi3(const pSwaggerFile, pOutputFolder: string);
var
    vSwagDoc: TSwagDoc;
begin
    vSwagDoc := TSwagDoc.Create;
    try
        vSwagDoc.LoadFromFile(pSwaggerFile); // SpecVersion pasa a ser svSwagger2
        vSwagDoc.SpecVersion := svOpenApi3;
        vSwagDoc.GenerateSwaggerJson;
        vSwagDoc.SwaggerFilesFolder := pOutputFolder;
        vSwagDoc.SaveSwaggerJsonToFile; // openapi.json
    finally
        vSwagDoc.Free;
    end;
end;

```

La dirección opuesta también funciona. Cuando se carga un documento OpenAPI 3, `Host`, `BasePath` y `Schemes` se completan a partir del primer servidor, reemplazando sus variables por sus valores predeterminados.

6.14 Definiciones de seguridad personalizadas

Consulte la sección 4.3. Un esquema que tiene una representación diferente en OpenAPI 3 sobrescribe `GenerateJsonObject(pVersion)`:

```

function TMySecurityDefinition.GenerateJsonObject(const pVersion: TSwagVersion): TJSONObject;
begin
    if pVersion <> svOpenApi3 then
        Exit(GenerateJsonObject);

    Result := TJSONObject.Create;
    Result.AddPair('type', 'http');
    Result.AddPair('scheme', 'digest');
    if not fDescription.IsEmpty then
        Result.AddPair('description', fDescription);
end;

```

6.15 Publicar ambas familias a partir del mismo modelo

```

procedure TApiDocumentation.Publish(const pRootFolder: string);
begin
    fSwagDoc.SpecVersion := svSwagger2;
    fSwagDoc.GenerateSwaggerJson;
    fSwagDoc.SwaggerFilesFolder := TPath.Combine(pRootFolder, 'v2');
    fSwagDoc.SaveSwaggerJsonToFile;           // v2\swagger.json

    fSwagDoc.SpecVersion := svOpenApi3;
    fSwagDoc.GenerateSwaggerJson;
    fSwagDoc.SwaggerFilesFolder := TPath.Combine(pRootFolder, 'v3');
    fSwagDoc.SaveSwaggerJsonToFile;           // v3\openapi.json
end;

```

TPath está declarado en System.IOUtils. Mantenga Host, BasePath y Schemes en el modelo para el documento Swagger 2.0 cuando use Servers.

7. Qué no recibe un documento Swagger 2.0

Cuando el modelo usa objetos de OpenAPI 3 y el documento se genera como Swagger 2.0, SwagDoc escribe un documento Swagger 2.0 válido traduciendo u omitiendo lo que Swagger 2.0 no puede representar:

Modelo de objetos	Comportamiento en Swagger 2.0
<code>RequestBody</code>	Parámetro body, o parámetros formData para contenidos de formulario y multipart
<code>Content</code> de una respuesta	Esquema y primer ejemplo del primer tipo de medio
Esquemas HTTP, OpenID Connect	API key en el encabezado <code>Authorization</code> (sección 6.9)
OAuth2 con varios flujos	Primer flujo que existe en Swagger 2.0
Requisitos de seguridad con esquemas no admitidos	Se eliminan
Parámetros cookie y querystring	No se escriben
Método QUERY, operaciones adicionales	No se escriben
Webhooks, callbacks, links	No se escriben
Ejemplos, encabezados, links, callbacks y tipos de medio reutilizables	No se escriben
Path items reutilizables referenciados por un path	Se escriben en línea en el path
<code>Servers</code>	No se escriben; se usan <code>Host</code> , <code>BasePath</code> y <code>Schemes</code>
<code>SelfUri</code> , <code>JsonSchemaDialect</code> , <code>summary</code> , <code>parent</code> y <code>kind</code> de las etiquetas, nombre del servidor, <code>summary</code> de la respuesta	No se escriben
Palabras clave de JSON Schema 2020-12	Se convierten como se describe en la sección 6.10

8. Despliegue

8.1 Archivos de la biblioteca

SwaggerDoc depende solo de la RTL de Delphi (`System.JSON`, `System.Generics.Collections`, `System.RegularExpressions` y units relacionadas). Se compiló y probó con Delphi 12 (Studio 23.0). Hay dos formas de usarla:

- **Library path.** Agregue la carpeta `Source` al library path o a la ruta de búsqueda del proyecto. Las units nuevas se encuentran automáticamente.
- **Package en tiempo de ejecución.** Instale `Source\SwaggerDoc.dpk`. Después de actualizar, recompile el package y elimine los archivos `.dcu` y `.bpl` antiguos, para que se compilen las units nuevas.

Units de la versión actual (N marca las units que son nuevas en esta versión):

Grupo	Units
Construct or de JSON Schema	<code>Json.Common.Helpers</code> , <code>Json.Schema</code> , <code>Json.Schema.Common.Types</code> , <code>Json.Schema.Field</code> , <code>Json.Schema.Field.Arrays</code> , <code>Json.Schema.Field.Booleans</code> , <code>Json.Schema.Field.Datetimes</code> , <code>Json.Schema.Field.Enums</code> , <code>Json.Schema.Field.Numbers</code> , <code>Json.Schema.Field.Objects</code> , <code>Json.Schema.Field.Strings</code>
Comunes	<code>Swag.Common.Consts</code> , <code>Swag.Common.Types</code> , <code>Swag.Common.Types.Helpers</code> , <code>Swag.Common.Json</code> (N)
Document o	<code>Swag.Doc</code> , <code>Swag.Doc.Info</code> , <code>Swag.Doc.Info.Contact</code> , <code>Swag.Doc.Info.License</code> , <code>Swag.Doc.Tags</code> , <code>Swag.Doc.Definition</code> , <code>Swag.Doc.Server</code> (N), <code>Swag.Doc.Extensions</code> (N), <code>Swag.Doc.Example</code> (N), <code>Swag.Doc.Link</code> (N)
Paths y operacion es	<code>Swag.Doc.Path</code> (alias), <code>Swag.Doc.Path.Operation</code> , <code>Swag.Doc.Path.Operation.RequestParameter</code> , <code>Swag.Doc.Path.Operation.RequestBody</code> (N), <code>Swag.Doc.Path.Operation.Response</code> , <code>Swag.Doc.Path.Operation.ResponseHeaders</code> (alias), <code>Swag.Doc.Path.Operation.Content</code> (N)
Seguridad	<code>Swag.Doc.SecurityDefinition</code> , <code>Swag.Doc.SecurityDefinitionBasic</code> , <code>Swag.Doc.SecurityDefinitionApiKey</code> , <code>Swag.Doc.SecurityDefinitionOAuth2</code> , <code>Swag.Doc.SecurityDefinitionHttp</code> (N), <code>Swag.Doc.SecurityDefinitionOpenIdConnect</code> (N), <code>Swag.Doc.SecurityDefinitionMutualTls</code> (N), <code>Swag.Doc.SecurityRequirement</code> (N)
Lectura, escritura y conversió n	<code>Swag.Doc.FileLoader</code> , <code>Swag.Doc.JsonConverter</code> (N), <code>Swag.Doc.OpenApi.Generator</code> (N), <code>Swag.Doc.OpenApi.Loader</code> (N)

Las clases de definición de seguridad se registran a sí mismas en su sección `initialization`. `LoadFromFile` las encuentra solo cuando sus units están enlazadas, lo cual ya garantiza `Swag.Doc.FileLoader`. Mantenga las units en el proyecto cuando las units de SwaggerDoc se enumeren explícitamente.

8.2 Archivos de Swagger UI

La carpeta `Deploy` tiene una carpeta por familia. Copie la carpeta de la familia que publica al servidor web y agregue el documento generado.

Deploy\Swagger2 - Swagger UI 3.3.1, para swagger.json

Archivo	Obligatorio	Función
index.html	Sí	Página y configuración. La opción <code>url</code> es <code>/api/help/swagger.json</code> ; cámbiela por la dirección de su documento
swagger-ui.css	Sí	Estilos
swagger-ui-bundle.js	Sí	Swagger UI
swagger-ui-standalone-preset.js	Sí	Diseño de la barra superior
oauth2-redirect.html	Para OAuth2	Recibe el token del diálogo Authorize
favicon-16x16.png , favicon-32x32.png	No	Íconos
swagger-ui.js	No	Versión en módulo de Swagger UI, no utilizada por <code>index.html</code>
*.map	No	Source maps, usados solo para depurar Swagger UI
swagger.json	Sí	Generado por SwagDoc con <code>svSwagger2</code>
readme.txt	No	Instrucciones

La página también carga fuentes desde `fonts.googleapis.com`. Sin acceso a internet, la página recurre a fuentes locales.

Deploy\OpenApi3 - Swagger UI 5.32.15, para openapi.json

Archivo	Obligatorio	Función
index.html	Sí	Página
index.css	Sí	Estilos de la página
swagger-initializer.js	Sí	Configuración. La opción <code>url</code> es <code>./openapi.json</code>
swagger-ui.css	Sí	Estilos
swagger-ui-bundle.js	Sí	Swagger UI
swagger-ui-standalone-preset.js	Sí	Diseño de la barra superior
oauth2-redirect.html	Para OAuth2	Recibe el token del diálogo Authorize
favicon-16x16.png , favicon-32x32.png	No	Íconos
LICENSE , NOTICE , *.LICENSE.txt	Al redistribuir	Avisos de la Apache License 2.0 de Swagger UI
openapi.json	Sí	Generado por SwagDoc con <code>svOpenApi3</code>
readme.txt	No	Instrucciones

Swagger UI 3.x no puede mostrar documentos OpenAPI 3.1 ni 3.2. Publique `openapi.json` con Swagger UI 5 (los archivos de `Deploy\OpenApi3`, o una versión 5.x más reciente). Swagger UI 5 también muestra documentos Swagger 2.0.

8.3 Configuración de la página

swagger-initializer.js de Deploy\OpenApi3:

```
window.onload = function() {
  window.ui = SwaggerUIBundle({
    url: "/api/help/openapi.json",          // dirección del documento en su servidor
    validatorUrl: null,
    dom_id: '#swagger-ui',
    deepLinking: true,
    presets: [SwaggerUIBundle.presets.apis, SwaggerUIStandalonePreset],
    plugins: [SwaggerUIBundle.plugins.DownloadUrl],
    layout: "StandaloneLayout"
  });
};
```

Para ofrecer ambos documentos en una sola página de Swagger UI 5, reemplace `url` por `urls`:

```
urls: [
  { url: "/api/help/openapi.json", name: "OpenAPI 3.2.1" },
  { url: "/api/help/swagger.json", name: "Swagger 2.0" }
],
"urls.primaryName": "OpenAPI 3.2.1",
```

8.4 Checklist del servidor web

- Sirva la página por HTTP o HTTPS. Los navegadores bloquean la solicitud que carga el documento cuando `index.html` se abre desde el sistema de archivos.
- Sirva los archivos `.json` con `Content-Type: application/json` y UTF-8.
- Cuando el documento se sirve desde un origen distinto al de la página, envíe los encabezados CORS (`Access-Control-Allow-Origin`).
- Regenera el documento cuando la API cambie (al iniciar o en la compilación) y evite cachés obsoletas: envíe `Cache-Control: no-cache` para el documento o agregue una versión a su URL.
- Para OAuth2, registre la dirección de `oauth2-redirect.html` como URI de redirección en el servidor de autorización.
- Proteja la ruta de la documentación cuando la API no sea pública.

8.5 Estructura de transición

/api/help/v2/	archivos de Deploy\Swagger2 + swagger.json	(consumidores existentes)
/api/help/v3/	archivos de Deploy\OpenApi3 + openapi.json	(consumidores nuevos)

Mantenga ambas hasta que los consumidores (generadores de clientes, API gateways, pruebas de contrato) lean el documento OpenAPI 3; después, elimine la ruta v2.

8.6 Cambios en la estructura del repositorio

Versión anterior	Versión actual
Deploy\index.html, Deploy\swagger.json y los archivos de Swagger UI en la raíz de Deploy	Deploy\Swagger2
La demo SampleApi escribe su ejecutable y swagger.json en Deploy	Los escribe en Deploy\Swagger2
ninguno	Deploy\OpenApi3 con Swagger UI 5.32.15 y openapi.json
ninguno	Demos\SampleOpenApi3 : la versión OpenAPI 3.2.1 de SampleApi, que escribe en Deploy\OpenApi3

8.7 Publicación del documento por la propia aplicación

Una API escrita con un framework web publica el documento desde su propio servidor, en lugar de copiar un archivo a un servidor web. La carpeta `Integrations` del repositorio tiene una página por framework.

Con [Horse](#), el middleware de `Integrations\Horse` publica el documento y la página que lo presenta. Una aplicación que ya lo utiliza cambia una propiedad:

```
THorse.Use(HorseSwagDoc);

SwagDocApi.SpecVersion := svOpenApi3;
SwagDocConfig.DocumentRoute := '/docs/openapi.json';
```

Con [DelphiMVCFramework](#), SwagDoc se distribuye dentro de la carpeta `lib/swagdoc` del framework y el middleware `MVCFramework.Middleware.Swagger` construye el documento en cada solicitud. Publicar un documento OpenAPI 3 requiere una versión distribuida que declare `TSwagVersion` y que el middleware defina `SpecVersion := svOpenApi3`.

La página que presenta el documento también necesita entender la familia. Los archivos de Swagger UI publicados por una aplicación escrita para Swagger 2.0 se reemplazan por la distribución de `Deploy\OpenApi3`.

9. Validación y pruebas

9.1 Regresión de Swagger 2.0

Genere `swagger.json` con la versión anterior y con la versión actual, sin cambiar `SpecVersion`, y compare los archivos:

```
fc /b before\swagger.json after\swagger.json
```

Los documentos construidos en código deben ser idénticos. Los documentos cargados desde archivos pueden diferir según lo descrito en la sección 4.2.

9.2 Validación de OpenAPI 3.2.1

- Valide `openapi.json` contra el JSON Schema de la especificación OpenAPI 3.2 publicado en <https://spec.openapis.org>, con cualquier validador de JSON Schema 2020-12.
- Abra el documento en Swagger UI 5 y verifique las operaciones, los esquemas, el diálogo Authorize y las solicitudes de Try it out.
- Cargue el archivo generado con `LoadFromFile` y génerele nuevamente: el resultado debe ser idéntico al archivo original.

9.3 Validación de Swagger 2.0

- Valide `swagger.json` en Swagger Editor o con un validador de Swagger 2.0.
- Delphi escribe la barra de las cadenas JSON como `\`. Es JSON válido, pero las herramientas que analizan el texto como YAML (Swagger Editor 4, por ejemplo) informan "unknown escape character". Reformatee el JSON antes de pegarlo en esas herramientas. La versión anterior tenía la misma salida.

10. Limitaciones conocidas

- Swagger UI 5.32.15 no enumera los webhooks de los documentos OpenAPI 3.2 (solo los de documentos 3.1) y no muestra `additionalOperations`. Ambos están presentes en `openapi.json`.
- Swagger UI 5.32.15 muestra una advertencia cuando `jsonSchemaDialect` es diferente de `https://spec.openapis.org/oas/3.1/dialect/base`. Deje `JsonSchemaDialect` vacío, a menos que los esquemas usen otro dialecto.
- La conversión a Swagger 2.0 omite los objetos enumerados en la sección 7.
- `Servers` no se convierte a `host`, `basePath` y `schemes` cuando un documento construido en código se genera como Swagger 2.0.
- `LoadFromFile` no puede leer una clase de definición de seguridad personalizada, porque los tipos de esquema son una enumeración cerrada.

11. Checklist de migración

- ☐ Código fuente de SwagDoc actualizado, package recompilado y archivos `.dcu` y `.bp1` antiguos eliminados.
- ☐ Arreglos, bucles y sentencias `case` sobre las enumeraciones ampliadas revisados.
- ☐ Definiciones de seguridad personalizadas actualizadas con `overload` y `SupportsVersion`.
- ☐ Units nuevas agregadas a los packages que enumeran las units de SwagDoc.
- ☐ Salida Swagger 2.0 comparada con la versión anterior.
- ☐ `SpecVersion := svOpenApi3` asignado, y `SwaggerFileName` asignado si la URL no debe cambiar.
- ☐ `openapi.json` validado contra el esquema de OpenAPI 3.2.
- ☐ Archivos de Swagger UI 5 de `Deploy\OpenApi3` publicados y `swagger-initializer.js` apuntando al documento.
- ☐ Servidor web sirviendo JSON con el tipo de contenido correcto y los encabezados de CORS y caché.
- ☐ URI de redirección de OAuth2 registrada para `oauth2-redirect.html`.
- ☐ Scripts actualizados para las carpetas `Deploy\Swagger2` y `Deploy\OpenApi3`.
- ☐ `Host`, `BasePath` y `Schemes` mantenidos si `swagger.json` todavía se publica.
- ☐ Requisitos de seguridad declarados explícitamente, con ámbitos.
- ☐ Consumidores de la API informados y migrados al documento OpenAPI 3.

12. Referencias

- Especificación Swagger 2.0: <https://github.com/OAI/OpenAPI-Specification/blob/main/versions/2.0.md>
- Especificación OpenAPI 3.2.1: <https://github.com/OAI/OpenAPI-Specification/blob/main/versions/3.2.1.md>
- JSON Schema 2020-12: <https://json-schema.org/draft/2020-12>
- Distribución de Swagger UI: <https://github.com/swagger-api/swagger-ui/tree/master/dist>
- Repositorio de SwagDoc: <https://github.com/marcelojaloto/SwagDoc>
- Demos de SwagDoc: `Demos\SampleApi` (Swagger 2.0) y `Demos\SampleOpenApi3` (OpenAPI 3.2.1)
- Aplicaciones de ejemplo que publican un documento OpenAPI 3.2.1:
<https://github.com/marcelojaloto/Delphi/tree/master/samples> — `server-api-rest-dmvc`
(DelphiMVCFramework) y `tasks-manager-horse` (Horse)